# Software Requirements Specification

## Supply Chain Management Simulator

Software Engineering Mini Project

| | |
|---|---|
| Document ID | SCM-SRS-001 |
| Project | Supply Chain Management Simulator |
| Version | 1.0 |
| Authors | Siri Krishna C R, Spoorthi Poornachandra, D R Rushmitha, Kallepalli Sai Meghana, Shraddha |
| Date | 11-10-2026 |
| Status | Draft |
| Related documents | Software Architecture and Design Document v1.0 (SCM-SAD-001), Software Test Plan v1.0 |

## Revision history

| Version | Date | Author | Change summary | Approval |
|---|---|---|---|---|
| 1.0 | 11-10-2026 | Project team | Initial SRS: 24 functional, 5 non-functional and 5 security requirements, two use-case diagrams, requirements traceability matrix | |

## Approvals

| Role | Name | Signature / Email | Date |
|---|---|---|---|
| Course Coordinator | | | |
| Repository owner and scrum master | Kallepalli Sai Meghana | | |
| QA lead | D R Rushmitha | | |
| Developer | Siri Krishna C R | | |
| Developer | Spoorthi Poornachandra | | |
| Developer | Shraddha | | |

# 1. Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, non-functional and security requirements of the Supply Chain Management Simulator (SCM Simulator), together with its external interfaces and the criteria used to verify each requirement. It is the baseline for design (SAD v1.0), testing (Software Test Plan v1.0) and acceptance.

## 1.2 Scope

The SCM Simulator is a command-line application that simulates, one day at a time, a single warehouse that sells multiple products to customers and replenishes its stock from multiple suppliers. It covers customer demand generation, order fulfilment with backorders, demand forecasting, replenishment decisions (safety stock, reorder point and economic order quantity), supplier selection, purchase orders and supplier delays, inbound transport cost, cost accounting, performance indicators and reporting to the console and to CSV.

Out of scope: networks with more than one warehouse, integration with real ERP or supplier systems, a graphical user interface, a persistent database, and user accounts or authentication. All data is simulated from a scenario configuration file.

## 1.3 Audience

Developers, QA engineers, reviewers and course evaluators.

## 1.4 Definitions

**Table 1: Acronyms and terms**

| Term | Definition |
|---|---|
| Backorder | Customer demand accepted but not yet shipped because stock was insufficient |
| CI | Continuous Integration: automated tests run on every pull request and push to main |

| Term | Definition |
|---|---|
| CLI | Command-Line Interface |
| CSV | Comma-Separated Values file |
| EOQ | Economic Order Quantity: order size that balances ordering and holding cost |
| FIFO | First In, First Out: older customer orders are served first |
| KPI | Key Performance Indicator, for example fill rate or service level |
| Lead time | Days between placing a purchase order and its expected delivery |
| MAE / MAPE | Mean Absolute Error / Mean Absolute Percentage Error of a forecast |
| PO | Purchase Order placed with a supplier |
| ROP / SS | Reorder Point / Safety Stock |
| RTM | Requirements Traceability Matrix |
| SAD | Software Architecture and Design Document |

### 1.5 References


1. ISO/IEC/IEEE 29148:2018, Systems and software engineering: Life cycle processes, Requirements engineering. https://www.iso.org/standard/72089.html
2. Software Architecture and Design Document, Supply Chain Management Simulator, v1.0 (SCM-SAD-001).
3. Software Test Plan, Supply Chain Management Simulator, v1.0.
4. OWASP, CSV Injection. https://owasp.org/www-community/attacks/CSV_Injection


### 1.6 Document overview

Section 2 describes the product, its users, constraints, assumptions and how requirements are allocated to increments. Section 3 specifies external interfaces. Section 4 states the functional requirements and Section 5 the non-functional, security, compliance and design constraints. Section 6 gives acceptance criteria, Section 7 the use-case models, and Section 8 the traceability matrix with the verification method for each requirement. "Shall" marks a mandatory requirement. Every requirement has a unique, permanent ID; changes are recorded in the revision history with a new document version.

## 2. Overall description

### 2.1 Product perspective

The SCM Simulator is a standalone Python application run from a terminal. It reads a scenario from a JSON configuration file, simulates the warehouse in memory for the requested number of days, prints a summary and writes day-by-day results to a CSV file. It has no network connection at runtime and no external system dependencies beyond the Python 3.12 standard library.

### 2.2 Major product functions

- Load and validate a scenario configuration (products, suppliers, demand, policy, transport).
- Generate daily customer demand per product using constant, uniform random or seasonal patterns.
- Fulfil customer orders oldest first and carry unmet demand as backorders.
- Forecast demand by moving average or exponential smoothing.

- Decide when and how much to reorder using safety stock, reorder point and EOQ.
- Select a supplier by cheapest price, shortest lead time or a balanced score.
- Place purchase orders, track their status and apply reliability-based delivery delays.
- Account for holding, stockout, ordering, purchase and transport cost.
- Report fill rate, service level, average inventory, inventory turnover and forecast error; export results to CSV.

### 2.3 User roles and characteristics

| Role | Description | Main needs |
|---|---|---|
| Supply Chain Analyst | Runs simulations and interprets results. Familiar with inventory concepts; comfortable with a terminal. | Repeatable runs, clear summary, CSV for further analysis |
| Scenario Planner | Prepares scenario files: products, suppliers, demand and policy settings. | Change the scenario without touching code; clear messages when a value is wrong |

### 2.4 Operating environment

Windows 10 or later, macOS 13 or later, or Ubuntu 22.04 or later, with Python 3.12. Automated tests run on GitHub Actions (ubuntu-latest).

### 2.5 Constraints

- Python 3.12 standard library only at runtime; pytest is used for testing.
- Command-line interface only; single process; no database.
- One warehouse; time advances in whole days.
- Development through a shared GitHub repository with a protected main branch, pull requests with at least one approval, and CI that must pass before merge.

### 2.6 Assumptions and dependencies

- All scenario values (costs, prices, lead times, demand parameters) are supplied by the configuration file and are fictional.
- Supplier lead time already includes transport time; transport affects cost only.
- Unmet demand is backordered, not lost.
- A run with the same configuration and seed must always produce the same results.

### 2.7 Apportioning of requirements

Requirements are delivered in two increments. Increment 1 produces a minimum working simulator that runs end to end from the command line. Increment 2 completes the product. No requirement is deferred beyond Increment 2.

| Increment | Goal | Requirements |
|---|---|---|
| Increment 1 | Minimum working simulator: load a scenario, run it day by day, print results per product | SCM-F-001 to 005, SCM-F-010 to 014, SCM-F-020 to 024, SCM-F-030, SCM-F-031, SCM-F-035 (engine and CLI without costs, delays, transport and KPIs), SCM-SR-001, SCM-SR-002 |
| Increment 2 | Complete product: costs, supplier delays, transport cost, second forecasting method, KPIs, CSV export, full CLI | SCM-F-006, SCM-F-015, SCM-F-025, SCM-F-032 to 034, SCM-F-035 (complete), SCM-NF-001 to 005, SCM-SR-003 |
| Throughout | Process and repository controls | SCM-SR-004, SCM-SR-005 |

# 3. External interface requirements

## 3.1 User interfaces

The user interacts through a command line:

| Element | Specification |
|---|---|
| Command | python -m src.main [--config PATH] [--days N] [--out PATH] |
| --config | Scenario file; default data/sample_config.json |
| --days | Overrides the configured number of days; integer from 1 to 3650 |
| --out | CSV output path; default output/results.csv |
| --help | Prints usage |
| Output | Plain-text summary of costs (Rs, 2 decimals) and KPIs (percent, 2 decimals); CSV file with one row per product per day |
| Errors | One line on standard error starting with "Error:", naming the file or field at fault; no stack trace |

## 3.2 Hardware interfaces

None beyond a standard computer able to run Python 3.12.

## 3.3 Software interfaces

| Interface | Direction | Description |
|---|---|---|
| Scenario configuration (JSON) | Input | Sections: simulation, policy, warehouse, products, suppliers, demand, transport_modes. Maximum file size 1 MB. |
| Daily results (CSV) | Output | Columns: day, product_id, demand, shipped, stock_end, backorder, on_order, forecast. |
| Python 3.12 runtime | Platform | Standard library modules only (dataclasses, json, csv, math, random, statistics, argparse, pathlib). |
| GitHub Actions | Development | Runs the pytest suite on every pull request and every push to main. |

## 3.4 Communications

The application makes no network connections at runtime. Development traffic to GitHub uses HTTPS.

# 4. System features (detailed)

Each requirement includes acceptance criteria and a reference to its test cases in the Software Test Plan. IDs follow SCM-F-###. Numerical examples in the acceptance criteria are the expected values used by the tests.

## 4.1 Products, inventory and costs

Description: maintain product data, track warehouse stock per product, compute replenishment parameters and accumulate operating costs.

| Req ID | Requirement (shall...) | Type | Priority | Source / Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| SCM-F-001 | The system shall represent each product with an ID, name, unit cost, daily holding cost, stockout penalty and order cost, and shall reject a product whose ID or name is empty or whose cost values are negative. | Functional | High | Scenario Planner | AC-SCM-F-001: a valid configuration entry creates the product; an empty ID or a negative cost is rejected with a message naming the field. Test: TC-PRD-01 to 04 | None |
| SCM-F-002 | The system shall maintain a product catalog that rejects duplicate product IDs and lists products in ascending ID order. | Functional | High | Scenario Planner | AC-SCM-F-002: a duplicate ID is rejected; an unknown ID is reported as not found; products added as P-003, P-001, P-002 are listed P-001, P-002, P-003. Test: TC-PRD-05 to 10 | SCM-F-001 |
| SCM-F-003 | The system shall track the stock of each product in the warehouse, increasing it on receipt and decreasing it on shipment, and shall never let stock fall below zero. | Functional | High | Supply Chain Analyst | AC-SCM-F-003: with 5 units in stock, a request for 8 ships 5 and leaves 0; zero or negative quantities are rejected. Test: TC-INV-01 to 06 | None |
| SCM-F-004 | The system shall compute the economic order quantity as ceil(sqrt(2DS/H)) for annual demand D, order cost S and annual holding cost per unit H. | Functional | High | Supply Chain Analyst | AC-SCM-F-004: D=1000, S=50, H=2 gives 224; D=0 gives 0; H of 0 or less is rejected. Test: TC-INV-07 to 10 | None |
| SCM-F-005 | The system shall compute safety stock and reorder point and shall trigger replenishment when stock plus open purchase-order quantity is at or below the reorder point. | Functional | High | Supply Chain Analyst | AC-SCM-F-005: z=1.65, daily std 4, lead time 4 gives safety stock 14; demand 10, lead time 4, safety stock 14 gives reorder point 54; stock 50 with 10 on order does not trigger. Test: TC-INV-11 to 15 | SCM-F-003 |
| SCM-F-006 | The system shall accumulate holding, stockout, ordering, purchase and transport cost for the run and report each category and the total. | Functional | High | Supply Chain Analyst | AC-SCM-F-006: 10 units at Rs 0.50 per day add Rs 5.00 holding; 3 units short at Rs 30 add Rs 90.00 stockout; total equals the sum of the categories. Test: TC-CST-01 to 04 | SCM-F-001 |

## 4.2 Suppliers and purchase orders

Description: hold supplier data, choose a supplier for each replenishment, and manage purchase orders from placement to delivery, including late deliveries.

| Req ID | Requirement (shall...) | Type | Priority | Source / Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| SCM-F-010 | The system shall represent each supplier with an ID, name, lead time in days, unit price per product, reliability between 0 and 1 and maximum order quantity, and shall reject invalid values. | Functional | High | Scenario Planner | AC-SCM-F-010: lead time 0 is rejected; reliability 1.5 is rejected; asking a supplier for the price of a product it does not sell reports not found. Test: TC-SUP-01 to 05 | None |
| SCM-F-011 | The system shall keep a supplier registry and list, for any product, the suppliers that sell it. | Functional | High | Supply Chain Analyst | AC-SCM-F-011: with the sample scenario, P-002 returns S-001 and S-003; an unknown product returns an empty list. Test: TC-SUP-06 to 09 | SCM-F-010 |

| Req ID | Requirement (shall...) | Type | Priority | Source / Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| SCM-F-012 | The system shall select a supplier for a product using the configured strategy: cheapest price, shortest lead time, or a balanced score of price, lead time and reliability. | Functional | High | Supply Chain Analyst | AC-SCM-F-012: for P-001, cheapest gives S-001, fastest gives S-002, balanced gives S-002 (score 0.606 against 0.760); ties go to the lower supplier ID. Test: TC-SUP-10 to 15 | SCM-F-011 |
| SCM-F-013 | The system shall create a purchase order with the supplier's unit price and an expected delivery day equal to the order day plus the supplier lead time, rejecting quantities outside 1 to the supplier's maximum order quantity. | Functional | High | Supply Chain Analyst | AC-SCM-F-013: order day 5 with a 3-day lead time gives delivery day 8; 100 units of P-001 from S-001 cost Rs 11,000.00; 201 units from S-002 are rejected. Test: TC-PO-01 to 05 | SCM-F-010 |
| SCM-F-014 | The system shall manage purchase order status (PLACED, IN_TRANSIT, DELIVERED, CANCELLED), allow only defined transitions, and release an order for receipt on its expected delivery day. | Functional | High | Supply Chain Analyst | AC-SCM-F-014: DELIVERED to PLACED is rejected; an order due on day 8 is not released on day 7 and is released on day 8; delivered and cancelled orders are excluded from the on-order quantity. Test: TC-PO-06 to 10 | SCM-F-013 |
| SCM-F-015 | The system shall delay a purchase order by 1 to the configured maximum number of days with probability equal to one minus the supplier's reliability, using the run's seeded random generator. | Functional | Medium | Supply Chain Analyst | AC-SCM-F-015: reliability 1.0 never delays; reliability 0.0 always delays by 1 to 3 days; the same seed gives the same delays. Test: TC-PO-11 to 14 | SCM-F-014 |

## 4.3 Demand, customer orders and forecasting

Description: generate customer demand, record and fulfil customer orders oldest first, and forecast future demand.

| Req ID | Requirement (shall...) | Type | Priority | Source / Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| SCM-F-020 | The system shall generate daily demand for each product using the constant, uniform random or seasonal pattern defined in the configuration. | Functional | High | Scenario Planner | AC-SCM-F-020: constant base 4 gives 4 every day; uniform base 20, spread 5 stays between 15 and 25; seasonal base 8, amplitude 3, period 4 gives 8, 11, 8, 5 on days 0 to 3. Test: TC-DEM-01 to 05 | None |
| SCM-F-021 | The system shall record each customer order with its quantity, shipped quantity and status (PENDING, PARTIAL, FULFILLED). | Functional | High | Supply Chain Analyst | AC-SCM-F-021: an order of 10 with 4 shipped is PARTIAL with 6 remaining; shipping more than the remaining quantity is rejected. Test: TC-ORD-06 to 09 | None |
| SCM-F-022 | The system shall number customer orders sequentially and list the open orders of a product oldest first. | Functional | High | Supply Chain Analyst | AC-SCM-F-022: IDs are CO-0001, CO-0002 in creation order; a day-1 order is listed before a day-3 order; fulfilled orders are not listed. Test: TC-ORD-10 to 13 | SCM-F-021 |

| Req ID | Requirement (shall...) | Type | Priority | Source / Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| SCM-F-023 | The system shall allocate available stock to open orders oldest first and keep any unfilled quantity as a backorder. | Functional | High | Supply Chain Analyst | AC-SCM-F-023: orders of 5 and 5 with 7 units available leave the first FULFILLED, the second PARTIAL with 2 shipped, and a backorder of 3. Test: TC-ORD-14 to 17 | SCM-F-022 |
| SCM-F-024 | The system shall forecast daily demand as the moving average of the most recent window of days and compute the standard deviation of demand over the same window. | Functional | High | Supply Chain Analyst | AC-SCM-F-024: history 10, 20, 30, 40 with window 2 gives 35.0; a history shorter than the window uses all values; the standard deviation of 10, 20, 30, 40 is about 11.18. Test: TC-FC-01 to 05 | None |
| SCM-F-025 | The system shall offer exponential smoothing as an alternative forecasting method and report forecast error as MAE and MAPE. | Functional | Medium | Supply Chain Analyst | AC-SCM-F-025: history 10, 20, 30 with alpha 0.5 gives 22.5; actual 10, 20 against forecast 12, 18 gives MAE 2.0 and MAPE 15.0%. Test: TC-FC-06 to 10 | SCM-F-024 |

## 4.4 Simulation control, configuration, logistics and reporting

Description: load the scenario, run the day-by-day simulation, cost inbound shipments, compute KPIs and report results.

| Req ID | Requirement (shall...) | Type | Priority | Source / Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| SCM-F-030 | The system shall advance the simulation one day at a time for the configured number of days and record each demand, shipment, stockout and purchase-order event with its day. | Functional | High | Supply Chain Analyst | AC-SCM-F-030: a 3-day run ends after 3 steps and a further step is rejected; events can be filtered and counted by type. Test: TC-CLK-01 to 05 | None |
| SCM-F-031 | The system shall load the scenario from a JSON file and reject it, naming the problem, when a section is missing, a value is out of range or a reference is unknown. | Functional | High | Scenario Planner | AC-SCM-F-031: a missing "suppliers" section, demand for P-999, a product no supplier sells, 0 or 5000 days, and malformed JSON are each rejected with a message naming the cause. Test: TC-CFG-01 to 06 | SCM-SR-001, SCM-SR-002 |
| SCM-F-032 | The system shall compute inbound transport cost as distance in km times quantity times the configured cost per unit-km of the transport mode. | Functional | Medium | Supply Chain Analyst | AC-SCM-F-032: 350 km, 100 units by road at Rs 0.02 gives Rs 700.00; an unknown mode is rejected. Test: TC-SHP-01 to 04 | None |
| SCM-F-033 | The system shall compute fill rate, service level, average inventory and inventory turnover for each run. | Functional | High | Supply Chain Analyst | AC-SCM-F-033: 180 shipped of 200 demanded gives fill rate 0.90; 3 stockout days of 30 gives service level 0.90; COGS 1200 over average inventory value 400 gives turnover 3.0. Test: TC-KPI-01 to 05 | SCM-F-030 |
| SCM-F-034 | The system shall export one row per product per day to CSV and print a summary of costs and KPIs, neutralising spreadsheet formula characters in text cells. | Functional | High | Supply Chain Analyst | AC-SCM-F-034: the CSV has a header and one row per product-day; a cell "=HYPERLINK(1)" is written as "'=HYPERLINK(1)"; the summary shows "Fill rate: 90.00%" for 0.9. Test: TC-RPT-01 to 05 | SCM-SR-003 |

| Req ID | Requirement (shall...) | Type | Priority | Source / Stakeholder | Acceptance criteria / Test case ref | Comments / Dependencies |
|---|---|---|---|---|---|---|
| SCM-F-035 | The system shall provide a command-line interface that runs the full daily simulation from a configuration file, with options for the number of days and the output path. | Functional | High | Supply Chain Analyst | AC-SCM-F-035: 30 days of the sample scenario produce 90 daily records; two runs with the same seed give identical records; a missing configuration file returns exit code 2 with a one-line error. Test: TC-SIM-01 to 06 | All of 4.1 to 4.4 |

# 5. Non-functional requirements (detailed)

Each NFR is measurable and linked to a test in the Software Test Plan. IDs follow SCM-NF-###.

| Req ID | Requirement | Category | Priority | Acceptance criteria / Measurement |
|---|---|---|---|---|
| SCM-NF-001 | A 365-day run of the sample scenario shall complete in under 2 seconds on a laptop with a 4-core CPU and 8 GB RAM. | Performance | High | Measured with time.perf_counter over the full run. Test: TC-NFR-01 |
| SCM-NF-002 | The same configuration and seed shall always produce a byte-identical CSV output. | Reliability (repeatability) | High | Two runs compared byte for byte. Test: TC-NFR-02, TC-SIM-03 |
| SCM-NF-003 | Every functional requirement shall have at least one automated test, and the full test suite shall run in CI in under 3 minutes. | Maintainability (testability) | High | RTM shows a test for every SCM-F requirement; CI job duration from GitHub Actions. |
| SCM-NF-004 | The CLI shall provide --help, and every input error shall be shown as one line starting with "Error:", with exit code 2 and no stack trace. | Usability | Medium | Missing and invalid configuration runs. Test: TC-SIM-06 |
| SCM-NF-005 | The system shall run on Windows, macOS and Linux with Python 3.12 using only the standard library at runtime. | Portability | Medium | Imports in src/ checked against the standard library; suite passes on Ubuntu (CI) and Windows. Test: TC-NFR-03 |

## 5.1 Security

### 5.1.1 Security objectives

- SO-1 Integrity of simulation inputs and outputs: a malformed or malicious scenario file cannot crash a run or produce misleading results, and exported files cannot execute code in the spreadsheet application that opens them.
- SO-2 Integrity of the codebase and build pipeline: no change reaches the main branch without review by another team member and a passing automated test run.

### 5.1.2 Security requirements

| Req ID | Requirement (shall...) | Type | Priority | Acceptance criteria / Test case ref |
|---|---|---|---|---|
| SCM-SR-001 | The system shall validate every configuration value for type, range and cross-reference before the simulation starts, and reject an invalid configuration. | Security | High | Invalid configurations are rejected before day 0 with a message naming the field. Test: TC-CFG-02 to 06 |
| SCM-SR-002 | The system shall reject configuration files larger than 1 MB and runs longer than 3650 days. | Security | High | A 1.1 MB file and --days 5000 are both rejected. Test: TC-CFG-06 |
| SCM-SR-003 | The system shall prefix with an apostrophe any CSV text cell that starts with =, +, - or @. | Security | High | Formula-like text is neutralised; numbers are unchanged. Test: TC-RPT-03, TC-RPT-04 |

| Req ID | Requirement (shall...) | Type | Priority | Acceptance criteria / Test case ref |
|---|---|---|---|---|
| SCM-SR-004 | The main branch shall accept changes only through pull requests with at least one approval and a passing CI check, with force pushes, deletion and bypass disabled. | Security | High | Ruleset protect-main inspected; a direct push to main is rejected by GitHub. |
| SCM-SR-005 | The project shall pin dependency versions in requirements.txt and shall not commit secrets or environment files. | Security | Medium | requirements.txt uses exact versions; .gitignore covers .env and virtual environments; checked in pull request review. |

## 5.2 Other quality of service

| Attribute | Requirement |
|---|---|
| Reliability | Invalid input stops the run before day 0 (SCM-SR-001); a run either completes and writes the full CSV or exits with an error. Results are repeatable (SCM-NF-002). |
| Availability | Not applicable: the simulator is a command-line tool run on demand, with no service to keep available. |
| Observability | Every demand, shipment, stockout and purchase-order event is recorded with its day in the event log (SCM-F-030). CI logs on GitHub Actions record every test run. |

## 5.3 Compliance

- Documentation follows the course templates for SRS, SAD and Test Plan, which are based on IEEE 830 and ISO/IEC/IEEE 29148.
- The source code is released under the MIT License, kept in the repository's LICENSE file.
- The system processes no personal data, so no data-protection regulation applies.

## 5.4 Design and implementation constraints

| Area | Constraint |
|---|---|
| Installation | Python 3.12 virtual environment; install with pip install -r requirements.txt; no environment variables or secrets; uninstall by deleting the project folder. |
| Build and delivery | No compilation step. Every pull request and push to main runs the test suite on GitHub Actions; dependencies are pinned to exact versions; the final release is tagged v1.0 on main. |
| Distribution | Single machine, single process. Distributed through the team GitHub repository. |
| Maintainability | One module per component as in SAD 3.4; a docstring on every public class and function; tests for each module in tests/; the main branch accepts reviewed pull requests only. |
| Reusability | Domain modules (inventory, suppliers, orders, forecasting, KPIs) have no dependency on the engine or the CLI and can be used on their own. |
| Portability | Windows, macOS and Linux with Python 3.12, standard library only at runtime (SCM-NF-005). |
| Cost | No licence or hosting cost: Python, pytest and GitHub Actions on a public repository are free. |
| Deadline | Requirements, design and test plan baselined on 11-10-2026; Increment 1 and Increment 2 completed by 11-11-2026, as set by the course. |
| Proof of concept | Not required; the techniques used (EOQ, reorder point, moving average, exponential smoothing) are standard. |
| Change management | Every change goes through a pull request reviewed by another member and merged by the repository owner after CI passes. Differences from this SRS are recorded in a deviation log and in this document's revision history. |

### 5.5 AI/ML

Not applicable. The simulator contains no machine-learning models; forecasting uses fixed statistical formulas.

## 6. Quality attributes & acceptance tests

- Exit criteria for acceptance: all High-priority functional requirements implemented and verified, no failing NFR test, all security requirements verified, and the RTM showing every linked test case passed.
- Acceptance test suites: products and inventory; suppliers and purchase orders; demand, orders and forecasting; configuration and security; simulation and CLI; non-functional (performance, repeatability, portability).
- Quality attributes in priority order: correctness of calculations, repeatability, testability, usability of the CLI, portability.

## 7. System models and diagrams

### 7.1 UML use-case diagrams

Figure 1 shows how the Supply Chain Analyst runs and reviews a simulation. Figure 2 shows how the Scenario Planner prepares a scenario. Every configuration use case includes UC-1.3 Validate configuration.

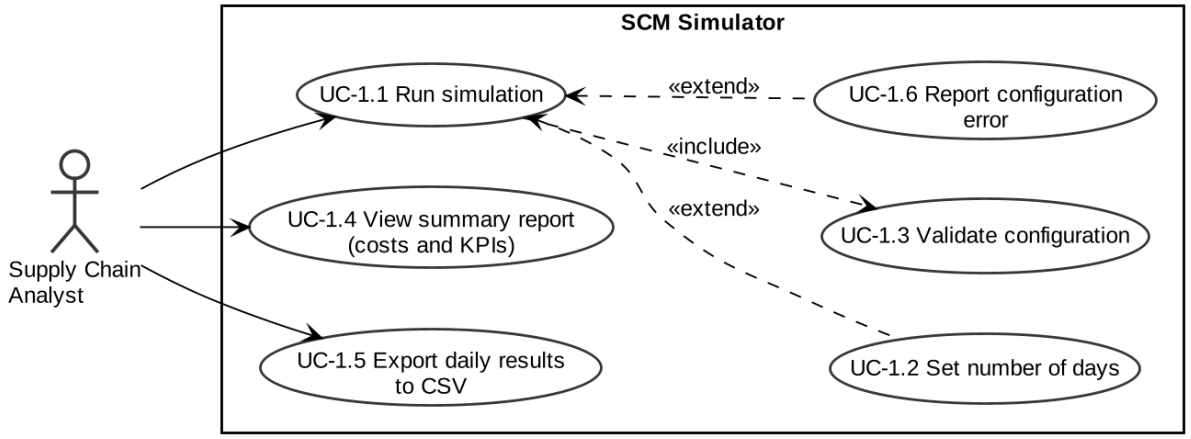



**Figure 1: Use-case diagram 1: Supply Chain Analyst**

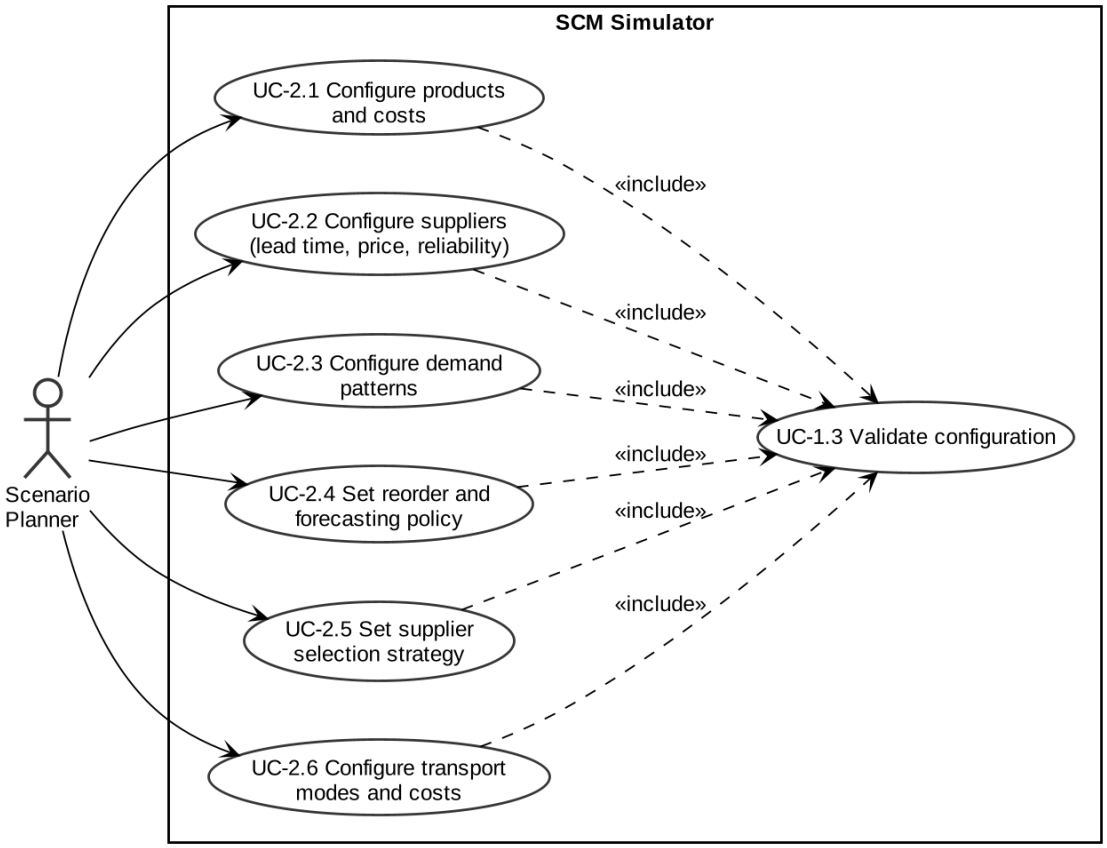



**Figure 2: Use-case diagram 2: Scenario Planner**

### 7.2 Main use-case descriptions

| Field | UC-1.1 Run simulation | UC-1.3 Validate configuration |
|---|---|---|
| Actor | Supply Chain Analyst | System (included by UC-1.1 and UC-2.1 to 2.6) |
| Precondition | A scenario file exists | A scenario file path is given |
| Main flow | 1. Analyst runs the command with optional --days and --out.<br>2. System validates the configuration (UC-1.3).<br>3. System simulates each day: receive deliveries, generate demand, fulfil orders, forecast, reorder.<br>4. System prints the summary and writes the CSV. | 1. Check file exists and is at most 1 MB.<br>2. Parse JSON.<br>3. Check required sections, value ranges and references. |
| Alternative flow | Configuration invalid: one-line error, exit code 2 (UC-1.6). | Any check fails: reject with a message naming the cause. |
| Postcondition | Summary shown; CSV written; exit code 0 | Validated configuration returned to the caller |
| Requirements | SCM-F-030, SCM-F-033 to 035, SCM-NF-001, 002, 004 | SCM-F-031, SCM-SR-001, SCM-SR-002 |

## 8. Requirements Traceability Matrix (RTM)

Status: N = not yet verified, P = partially verified, A = all linked test cases passed. Design references point to sections of SAD v1.0. The comments column gives the verification method: Test, Analysis, Inspection or Demonstration.

| Req ID | Requirement short | Section ref / Design spec | Module | Test case(s) | Status (N/P/A) | Comments |
|---|---|---|---|---|---|---|
| SCM-F-001 | Product model and validation | 4.1 / SAD 3.4 | products.py | TC-PRD-01 to 04 | N | Test |
| SCM-F-002 | Product catalog | 4.1 / SAD 3.4 | products.py | TC-PRD-05 to 10 | N | Test |
| SCM-F-003 | Warehouse stock | 4.1 / SAD 4.3.2 | inventory.py | TC-INV-01 to 06 | N | Test |
| SCM-F-004 | Economic order quantity | 4.1 / SAD 4.3.2 | inventory.py | TC-INV-07 to 10 | N | Test |
| SCM-F-005 | Safety stock and reorder point | 4.1 / SAD 4.3.2 | inventory.py | TC-INV-11 to 15 | N | Test |
| SCM-F-006 | Cost accounting | 4.1 / SAD 3.4 | costs.py | TC-CST-01 to 04 | N | Test |
| SCM-F-010 | Supplier model and validation | 4.2 / SAD 4.3.3 | suppliers.py | TC-SUP-01 to 05 | N | Test |
| SCM-F-011 | Supplier registry | 4.2 / SAD 4.3.3 | suppliers.py | TC-SUP-06 to 09 | N | Test |
| SCM-F-012 | Supplier selection | 4.2 / SAD 4.3.3 | suppliers.py | TC-SUP-10 to 15 | N | Test |
| SCM-F-013 | Purchase order creation | 4.2 / SAD 4.3.3 | purchase_orders.py | TC-PO-01 to 05 | N | Test |
| SCM-F-014 | PO lifecycle and order book | 4.2 / SAD 4.1, 4.3.3 | purchase_orders.py | TC-PO-06 to 10 | N | Test |
| SCM-F-015 | Supplier delivery delays | 4.2 / SAD 4.3.3 | purchase_orders.py | TC-PO-11 to 14 | N | Test |
| SCM-F-020 | Demand generation | 4.3 / SAD 3.4 | demand.py | TC-DEM-01 to 05 | N | Test |
| SCM-F-021 | Customer order model | 4.3 / SAD 4.3.4 | orders.py | TC-ORD-06 to 09 | N | Test |
| SCM-F-022 | Customer order book | 4.3 / SAD 4.3.4 | orders.py | TC-ORD-10 to 13 | N | Test |
| SCM-F-023 | FIFO allocation and backorders | 4.3 / SAD 4.3.4 | orders.py | TC-ORD-14 to 17 | N | Test |
| SCM-F-024 | Moving average forecast | 4.3 / SAD 4.1 | forecasting.py | TC-FC-01 to 05 | N | Test |
| SCM-F-025 | Exponential smoothing, MAE, MAPE | 4.3 / SAD 4.1 | forecasting.py | TC-FC-06 to 10 | N | Test |
| SCM-F-030 | Simulation clock and event log | 4.4 / SAD 4.4 | clock.py | TC-CLK-01 to 05 | N | Test |
| SCM-F-031 | Configuration loading and validation | 4.4 / SAD 4.3.5 | config_loader.py | TC-CFG-01 to 06 | N | Test |
| SCM-F-032 | Transport cost | 4.4 / SAD 3.4 | shipments.py | TC-SHP-01 to 04 | N | Test |
| SCM-F-033 | KPI computation | 4.4 / SAD 4.1 | kpis.py | TC-KPI-01 to 05 | N | Test |
| SCM-F-034 | CSV export and summary | 4.4 / SAD 4.5 | reports.py | TC-RPT-01 to 05 | N | Test |
| SCM-F-035 | Simulation engine and CLI | 4.4 / SAD 4.2, 4.3.1 | simulation.py, main.py | TC-SIM-01 to 06 | N | Test |
| SCM-NF-001 | Performance | 5 / SAD 3.1 | simulation.py | TC-NFR-01 | N | Test |
| SCM-NF-002 | Repeatability | 5 / SAD 3.5 (ADR-005) | simulation.py, demand.py, purchase_orders.py | TC-NFR-02, TC-SIM-03 | N | Test |
| SCM-NF-003 | Testability and CI time | 5 / SAD 3.6 | tests/, ci.yml | RTM review, CI duration | N | Analysis |
| SCM-NF-004 | CLI usability and errors | 5 / SAD 4.4, 4.5 | main.py | TC-SIM-06 | N | Test |
| SCM-NF-005 | Portability | 5 / SAD 3.5 (ADR-003) | All modules | TC-NFR-03 | N | Test, Inspection |
| SCM-SR-001 | Configuration validation | 5.1.2 / SAD 3.9 | config_loader.py | TC-CFG-02 to 06 | N | Test |
| SCM-SR-002 | Size and run-length limits | 5.1.2 / SAD 3.9 | config_loader.py | TC-CFG-06 | N | Test |
| SCM-SR-003 | CSV formula injection protection | 5.1.2 / SAD 3.9 | reports.py | TC-RPT-03, TC-RPT-04 | N | Test |
| SCM-SR-004 | Protected main branch | 5.1.2 / SAD 3.6, 3.9 | Repository settings | Ruleset inspection | N | Inspection, Demonstration |
| SCM-SR-005 | Pinned dependencies, no secrets | 5.1.2 / SAD 3.9 | requirements.txt, .gitignore | PR review | N | Inspection |